

instead of providing them analytically. Similarly, `dfpmin` very often succeeds when the gradient is calculated with finite differences. In our experience, this method often involves less total computation than one of the other methods that avoids analytic derivatives, such as `Powell1`.

To use this idea, all you need to do is supply a suitable functor to `dfpmin`, which remains unchanged. Here is the code, which is very similar to that of `Fdjac` in §9.7:

```
template <class T>
struct Funcc {
    Doub EPS;
    T &func;
    Doub f;
    Funcc(T &funcc) : EPS(1.0e-8), func(funcc) {}
    Doub operator() (VecDoub_I &x)
    {
        return f=func(x);
    }
};

void df(VecDoub_I &x, VecDoub_O &df)
{
    Int n=x.size();
    VecDoub xh=x;
    Doub fold=f;
    for (Int j=0;j<n;j++) {
        Doub temp=x[j];
        Doub h=EPS*abs(temp);
        if (h == 0.0) h=EPS;
        xh[j]=temp+h;
        h=xh[j]-temp;
        Doub fh=operator()(xh);
        xh[j]=temp;
        df[j]=(fh-fold)/h;
    }
};
```

quasinewton.h

Set to approximate square root of the machine precision.

Trick to reduce finite-precision error.

10.9.2 Advanced Implementations of Variable Metric Methods

Although rare, it can conceivably happen that roundoff errors cause the matrix $\mathbf{H}_i$ to become nearly singular or non-positive-definite. This can be serious, because the supposed search directions might then not lead downhill, and because nearly singular $\mathbf{H}_i$'s tend to give subsequent $\mathbf{H}_j$'s that are also nearly singular.

There is a simple fix for this rare problem, the same as was mentioned in §10.5: In case of any doubt, you should *restart* the algorithm at the claimed minimum point and see if it goes anywhere. Simple, but not very elegant. Modern implementations of quasi-Newton methods deal with the problem in a more sophisticated way.

Instead of building up an approximation to $\mathbf{A}^{-1}$, it is possible to build up an approximation of $\mathbf{A}$ itself. Then, instead of calculating the left-hand side of (10.9.4) directly, one solves the set of linear equations

$$\mathbf{A} \cdot (\mathbf{x} - \mathbf{x}_i) = -\nabla f(\mathbf{x}_i) \quad (10.9.11)$$

At first glance this seems like a bad idea, since solving (10.9.11) is a process of order N^3 — and anyway, how does this help the roundoff problem? The trick is not to store $\mathbf{A}$ but rather a triangular decomposition of $\mathbf{A}$, its *Cholesky decomposition* (cf. §2.9). The updating formula used for the Cholesky decomposition of $\mathbf{A}$ is of order N^2 and can be arranged to guarantee that the matrix remains positive-definite and nonsingular, even in the presence of finite roundoff. This method is due to Gill and Murray [1,2].

CITED REFERENCES AND FURTHER READING:

- Dennis, J.E., and Schnabel, R.B. 1983, *Numerical Methods for Unconstrained Optimization and Nonlinear Equations*; reprinted 1996 (Philadelphia: S.I.A.M.).[1]
 Jacobs, D.A.H. (ed.) 1977, *The State of the Art in Numerical Analysis* (London: Academic Press), Chapter III.1, §3 – §6 (by K. W. Brodlie).[2]
 Polak, E. 1971, *Computational Methods in Optimization* (New York: Academic Press), pp. 56ff.[3]
 Acton, F.S. 1970, *Numerical Methods That Work*; 1990, corrected edition (Washington, DC: Mathematical Association of America), pp. 467–468.

10.10 Linear Programming: The Simplex Method

The subject of *linear programming*, sometimes called *linear optimization*, concerns itself with the following problem: For n independent variables $x_1, \dots, x_n$, *minimize* the function

$$\zeta = c_1x_1 + c_2x_2 + \cdots + c_nx_n \quad (10.10.1)$$

subject to the nonnegativity conditions

$$x_1 \geq 0, \quad x_2 \geq 0, \quad \dots \quad x_n \geq 0 \quad (10.10.2)$$

and simultaneously subject to m additional constraints of the form

$$a_{i1}x_1 + a_{i2}x_2 + \cdots + a_{in}x_n \leq b_i \quad (10.10.3)$$

or

$$a_{i1}x_1 + a_{i2}x_2 + \cdots + a_{in}x_n = b_i \quad (10.10.4)$$

Here $i = 1, \dots, m$. Note that an inequality with a $\geq$ can be converted to a $\leq$ by multiplying by -1 . Some formulations of linear programming require you to write all the constraints with the b 's nonnegative and separately treat the $\geq$ and $\leq$ constraints. We will use the above formulation, with either sign of b_i , instead. However, it is still useful to refer to the inequalities with $b_i \leq 0$ as “ $\geq$ inequalities” (which they would be with $b_i \geq 0$), since, as we shall see, they enter the problem in a different way from the $\leq$ inequalities.

There is no particular significance in the number of constraints m being less than, equal to, or greater than the number of unknowns n . Also, note that there is no special significance to minimizing ζ in equation (10.10.1): We can convert a maximizing problem to a minimizing problem by changing the signs of all the c 's. The solution $x_1, \dots, x_n$ is the same, and the required maximum is the negative of the minimum ζ found.

A set of values $x_1, \dots, x_n$ that satisfies the constraints (10.10.2) – (10.10.4) is called a *feasible vector*. The function that we are trying to minimize is called the *objective function*. The feasible vector that minimizes the objective function is called the *optimal feasible vector*. An optimal feasible vector can fail to exist for two distinct reasons: (i) There are *no* feasible vectors, i.e., the given constraints

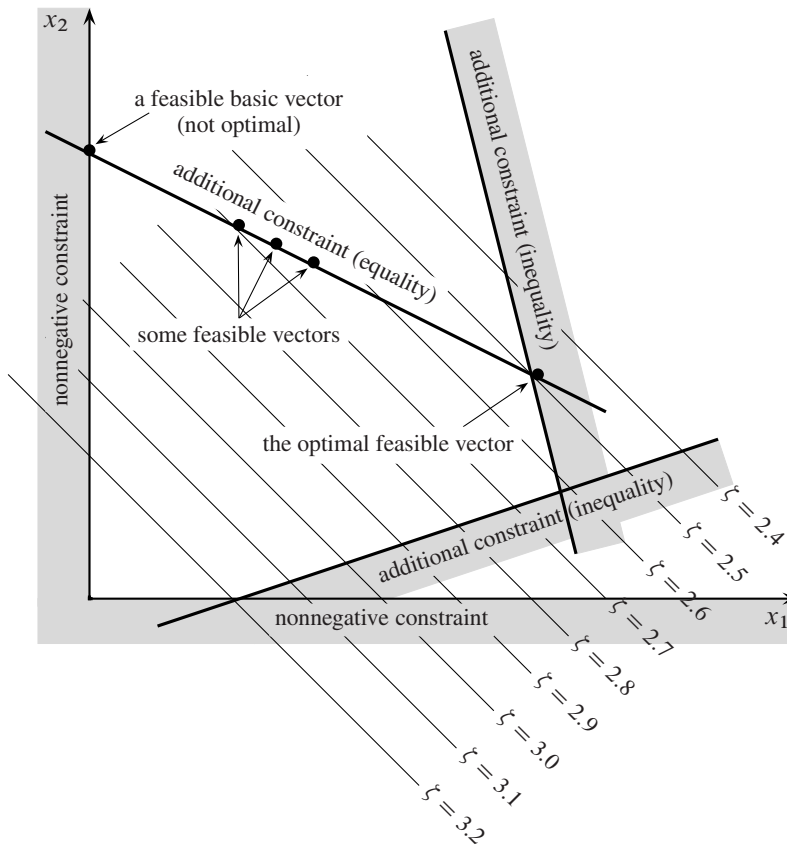


Figure 10.10.1. Basic concepts of linear programming. The case of only two independent variables, x_1 , x_2 , is shown. The linear function ζ , to be minimized, is represented by its contour lines. Nonnegativity constraints require x_1 and x_2 to be positive. Additional constraints may restrict the solution to regions (inequality constraints) or to surfaces of lower dimensionality (equality constraints). Feasible vectors satisfy all constraints. Feasible basic vectors also lie on the boundary of the allowed region. The simplex method steps among feasible basic vectors until the optimal feasible vector is found.

are incompatible, or (ii) there is no minimum, i.e., there is a direction in n -space where one or more of the variables can be taken to infinity while still satisfying the constraints, giving an unbounded value for the objective function. Figure 10.10.1 summarizes some of the terminology thus far.

As you see, the subject of linear programming is surrounded by notational and terminological thickets. Both of these thorny defenses are lovingly cultivated by a coterie of stern acolytes who have devoted themselves to the field. Actually, the basic ideas of linear programming are quite simple. Avoiding the shrubbery, let's elucidate the basics by means of a couple of specific examples; it should then be quite obvious how to generalize.

Why is linear programming so important? (i) Because “nonnegativity” is the usual constraint on any variable x_i that represents the tangible amount of some physical commodity, like guns, butter, dollars, units of vitamin E, food calories, kilowatt hours, mass, etc. Hence equation (10.10.2). (ii) Because one is often interested in additive (linear) limitations or bounds imposed by man or nature: minimum nutritional

requirement, maximum affordable cost, maximum on available labor or capital, minimum tolerable level of voter approval, etc. Hence equations (10.10.3) – (10.10.4). (iii) Because the function that one wants to optimize may be linear, or else may at least be approximated by a linear function — since that is the problem that linear programming *can* solve. Hence equation (10.10.1). For a short, semipopular survey of linear programming applications, see Bland [1].

10.10.1 Fundamental Theorem of Linear Optimization

Imagine that we start with a full n -dimensional space of candidate vectors. Then (in our mind's eye, at least) we carve away the regions that are eliminated in turn by each imposed constraint. Since the constraints are linear, every boundary introduced by this process is a plane, or rather a hyperplane. Equality constraints of the form (10.10.4) force the feasible region onto hyperplanes of smaller dimension, while inequalities simply divide the then-feasible region into allowed and nonallowed pieces.

When all the constraints are imposed, either we are left with some feasible region or else there are no feasible vectors. Since the feasible region is bounded by hyperplanes, it is geometrically a kind of convex polyhedron or simplex (cf. §10.5). If there is a feasible region, can the optimal feasible vector be somewhere in its interior, away from the boundaries? No, because the objective function is linear. This means that it always has a nonzero vector gradient. This, in turn, means that we could always decrease the objective function by running down the gradient until we hit a boundary wall.

The boundary of any geometrical region has one less dimension than its interior. Therefore, we can now run down the gradient projected into the boundary wall until we reach an edge of that wall. We can then run down that edge, and so on, down through whatever number of dimensions, until we finally arrive at a point, a *vertex* of the original simplex. Since this point has all n of its coordinates defined, it must be the solution of n simultaneous *equalities* drawn from the original set of equalities and inequalities (10.10.2) – (10.10.4).

Points that are feasible vectors and that satisfy n of the original constraints as equalities, including the nonnegativity constraints, are termed *feasible basic vectors*. If $n > m$, then a feasible basic vector has *at least* $n - m$ of its components equal to zero, since at least that many of the constraints (10.10.2) will be needed to make up the total of n . Put the other way, *at most* m components of a feasible basic vector are nonzero.

Put together the two preceding paragraphs and you have the *Fundamental Theorem of Linear Optimization*: If an optimal feasible vector exists, then there is a feasible basic vector that is optimal. (Didn't we warn you about the terminological thickets?)

The importance of the fundamental theorem is that it reduces the optimization problem to a “combinatorial” problem, that of determining which n constraints (out of the $m + n$ constraints in 10.10.2 – 10.10.4) should be satisfied by the optimal feasible vector. We have only to keep trying different combinations, and computing the objective function for each trial, until we find the best.

Doing this blindly would take halfway to forever. The *simplex method*, first published by Dantzig in 1948 (see [2]), is a way of organizing the procedure so that (i) a series of combinations is tried for which the objective function decreases at each step, and (ii) the optimal feasible vector is reached after a number of iterations that

is almost always no larger than of order m or n , whichever is larger. An interesting mathematical sidelight is that this second property, although known empirically ever since the simplex method was devised, was not proved to be true until the 1982 work of Stephen Smale. (For a contemporary account, see [3].)

10.10.2 Writing the General Problem in Standard Form

There is a standard form for linear programming problems, and we have to learn how to write a general problem like (10.10.1) – (10.10.4) in this standard form. For definiteness, consider the problem

$$\text{Minimize } \zeta = -40x_1 - 60x_2 \quad (10.10.5)$$

with the x 's nonnegative and also with

$$2x_1 + x_2 \leq 70 \quad (10.10.6)$$

$$x_1 + x_2 \geq 40 \quad (10.10.7)$$

$$x_1 + 3x_2 = 90 \quad (10.10.8)$$

First, we rewrite the inequalities as equalities. We do this by adding to the problem so-called *slack variables* $x_{n+1}, x_{n+2}, \dots$. In our example, equations (10.10.6) and (10.10.7) become

$$2x_1 + x_2 + x_3 = 70 \quad (10.10.9)$$

$$-x_1 - x_2 + x_4 = -40 \quad (10.10.10)$$

(A slack variable like x_4 for a $\geq$ inequality is sometimes called a *surplus* variable.) Requiring the slack variables to be nonnegative makes these equalities equivalent to the original inequalities. Once they are introduced, you treat slack variables on an equal footing with the original variables x_i ; then, at the very end, you just ignore them. The simplex solution for each slack variable is simply the amount by which the original inequality is satisfied.

The key idea in the simplex method is to start with a feasible basic vector and make a sequence of exchanges between basic and nonbasic variables. At each step the vector stays feasible (satisfies the constraints), and the objective function decreases (or at least does not increase).

How do we find an initial feasible basic vector to start the procedure? Suppose that our example were changed so that equations (10.10.7) and (10.10.8) were both $\leq$ inequalities, like (10.10.6). Then, after introducing slack variables, we would have

$$2x_1 + x_2 + x_3 = 70 \quad (10.10.11)$$

$$x_1 + x_2 + x_4 = 40 \quad (10.10.12)$$

$$x_1 + 3x_2 + x_5 = 90 \quad (10.10.13)$$

In this case it is easy to write down a feasible basic vector: Set the original variables x_1 and x_2 to zero and take $(x_3, x_4, x_5) = (70, 40, 90)$. Here $n = 2$ of the constraints, namely $x_1 \geq 0, x_2 \geq 0$, are satisfied as equalities, while $m = 3$ components of the feasible basic vector are nonzero. The variables (x_3, x_4, x_5) are called *basic variables*, while the variables that are zero, (x_1, x_2) , are called *nonbasic variables*. Note

that if we write equations (10.10.11) – (10.10.13) as a 3×5 matrix equation, then the last three columns of the matrix, corresponding to the slack variables (x_3, x_4, x_5), form a 3×3 unit matrix.

So $\leq$ constraints are easy. But how do we handle constraints like equations (10.10.7) and (10.10.8)? The trick is again to invent new variables called *artificial variables*. We rewrite equation (10.10.8) as

$$x_1 + 3x_2 + x_5 = 90 \quad (10.10.14)$$

Now equations (10.10.9), (10.10.10), and (10.10.14) are almost in the form to give us an easy initial feasible basic vector by setting $x_1 = x_2 = 0$. The obstacle is equation (10.10.10), which would give a negative value for x_4 . We have to precede the actual simplex procedure by a preliminary procedure, called *phase one* of the simplex method, to find an initial feasible vector. (The actual optimization is called *phase two*.)

In phase one, we replace our objective function (10.10.5) by a so-called *auxiliary objective function*,

$$\zeta' \equiv -x_4 \quad (10.10.15)$$

We now perform the simplex method on the auxiliary objective function (10.10.15) with the constraints (10.10.9), (10.10.10), and (10.10.14), starting with the basis given by $x_1 = x_2 = 0$. The variable x_4 starts off negative (at -40). Minimizing the function (10.10.15) drives x_4 toward satisfying $x_4 \geq 0$, the condition for feasibility. In fact, we don't even have to solve phase one all the way to the exact minimum. As we do the exchanges between variables during this phase, we continually redefine the auxiliary objective function at each iteration to be minus the sum of all negative basic variables. As soon as all the basic variables are nonnegative, we are done with phase one.

And what if the first phase *doesn't* drive the auxiliary objective function to a negative value (i.e., all basic variables nonnegative)? That signals that there is *no* initial feasible basic vector, i.e., that the constraints given to us are inconsistent among themselves. Report that fact, and you are done.

An artificial variable in an equality constraint is an example of a *zero variable*, a variable that must vanish in the optimal solution. Typically the way a zero variable gets to be zero is by being nonbasic in the optimal solution. So we can precede phase one with a “phase zero” in which we exchange each zero variable out of the basis.

One last piece of jargon: Slack and artificial variables are often called *logical* variables, to distinguish them from the original independent variables, which are sometimes called *structural* variables.

10.10.3 The Simplex Method: A Worked Example

The easiest way to describe the actual simplex procedure is with a worked example. We write the general linear programming problem in the following form: Minimize the objective function

$$\zeta = \mathbf{c} \cdot \mathbf{x} = c_1x_1 + c_2x_2 + \cdots + c_nx_n \quad (10.10.16)$$

subject to the constraints

$$\mathbf{A} \cdot \mathbf{x} = \mathbf{b} \quad (10.10.17)$$